UNIDADE 8 · VISÃO COMPUTACIONAL COM CNN

Rede convolucional na prática: gatos e cachorros

Treinar uma CNN no Keras e testá-la com fotos da internet

VISÃO GERAL

O que vamos fazer hoje

- Problema: dada a foto de um animal, dizer se é um gato ou um cachorro
- Treinar uma rede convolucional com 1000 fotos e testá-la com 400 que ela nunca viu
- Testar a rede com fotos da internet escolhidas pelos próprios alunos

Foco: mão na massa. Cada passo traz um código para executar e um resultado para interpretar.

VISÃO GERAL

Retomando: a receita do Keras

A receita é a mesma da aula anterior. Hoje mudam o passo 1 (os dados vêm de um arquivo) e o passo 3 (a lista de camadas).

1	2	3	4	5	6	7
Carregar os dados	**Preparar os dados**	**Montar o modelo**	**Compilar**	**Treinar**	**Avaliar**	**Prever**

Os passos em amarelo são os que mudam em relação ao MNIST.

VISÃO GERAL

O que muda em relação ao MNIST

	MNIST	Gatos e cachorros
Imagens	dígitos escritos à mão	fotos de gatos e cachorros
Tamanho	28 × 28 pixels	128 × 128 pixels (escolhido por nós)
Cor	cinza (1 canal)	colorida (3 canais)
Classes	10	2
Origem dos dados	carregados pelo Keras	arquivo .zip baixado da internet
Formato da entrada	(28, 28, 1)	(128, 128, 3)
Neurônios na saída	10	2

OS DADOS

O conjunto de dados

- Amostra do conjunto Dogs vs. Cats, do Kaggle, publicada no Zenodo
- 1000 imagens de treino e 400 de teste, em arquivos JPEG
- Arquivo .zip de 32,6 MB
- Licença CC BY 4.0: é preciso citar o autor

Citação: Jacquemont, M. Cats and Dogs sample. Zenodo, 2021. DOI 10.5281/zenodo.5226945

PASSO 1 · CARREGAR OS DADOS

Baixar o conjunto

```
import zipfile
import tensorflow as tf


url = "https://zenodo.org/records/5226945/files/cats_dogs_light.zip?download=1"
caminho = tf.keras.utils.get_file("cats_dogs_light.zip", origin=url, extract=False)


with zipfile.ZipFile(caminho) as z:
    nomes = z.namelist()


print(len(nomes), "arquivos")
print(sorted({"/".join(n.split("/")[:-1]) for n in nomes if "/" in n}))
```

Saída no Colab do professor: 1403 arquivos e as pastas cats_dogs_light/train e cats_dogs_light/test. O download levou cerca de 50 segundos.

PASSO 1 · CARREGAR OS DADOS

Conferir o conteúdo do arquivo

```
import os

arquivos = [n for n in nomes if not n.endswith("/")]

for parte in ["train", "test"]:
    nomes_parte = [os.path.basename(n) for n in arquivos if f"/{parte}/" in n]
    gatos = sum("cat" in n.lower() for n in nomes_parte)
    cachorros = sum("dog" in n.lower() for n in nomes_parte)
    print(parte, "->", len(nomes_parte), "arquivos | gatos:", gatos, "| cachorros:", cachorros)
```

Conjunto	Arquivos	Gatos	Cachorros
train (treino)	1000	455	545
test (teste)	400	200	200

O teste é equilibrado: um modelo que sempre respondesse a mesma classe acertaria 50%.

PASSO 1 · CARREGAR OS DADOS

A função de carregamento

```python
import os
import zipfile
import numpy as np
from PIL import Image


TAMANHO = (128, 128)


with zipfile.ZipFile(caminho) as z:
    z.extractall("dados")


def carregar_pasta(pasta):
    imagens, rotulos = [], []
    for nome in sorted(os.listdir(pasta)):
        if not nome.lower().endswith(".jpg"):
            continue
        img = Image.open(os.path.join(pasta, nome)).convert("RGB").resize(TAMANHO)
        imagens.append(np.array(img))
        rotulos.append(1 if "dog" in nome.lower() else 0)      # 0 = gato, 1 = cachorro
    return np.array(imagens), np.array(rotulos)
```

1. Abre cada arquivo .jpg
2. Converte para RGB
3. Redimensiona para 128 × 128
4. Guarda a imagem como array
5. O nome dá o rótulo: 0 gato, 1 cachorro

PASSO 1 · CARREGAR OS DADOS

Carregar treino e teste

```
x_treino, y_treino = carregar_pasta("dados/cats_dogs_light/train")
x_teste, y_teste = carregar_pasta("dados/cats_dogs_light/test")

print(x_treino.shape, y_treino.shape, x_teste.shape, y_teste.shape)
print("treino -> gatos:", (y_treino == 0).sum(), "| cachorros:", (y_treino == 1).sum())
```

Saída esperada: (1000, 128, 128, 3) (1000,) (400, 128, 128, 3) (400,) e treino -> gatos: 455 | cachorros: 545

- x_treino tem 1000 imagens de 128 × 128 pixels com 3 canais
- y_treino guarda 0 para gato e 1 para cachorro
- O mesmo vale para x_teste e y_teste, com 400 imagens

PASSO 1 · CARREGAR OS DADOS

Por que redimensionar e converter para RGB

Redimensionar

A rede exige que todas as imagens tenham o mesmo tamanho, e as fotos originais têm tamanhos diferentes

Escolhemos 128 × 128: imagens maiores deixam o treino mais lento; menores perdem detalhe

Converter para RGB

Uma foto pode vir em tons de cinza ou com transparência

convert("RGB") garante 3 canais (vermelho, verde e azul) em todas as imagens

O mesmo ajuste é aplicado a todas as imagens, do treino e do teste.

PASSO 1 · CARREGAR OS DADOS

Ver as imagens

```
import matplotlib.pyplot as plt


nomes_classes = ["gato", "cachorro"]


fig, eixos = plt.subplots(2, 5, figsize=(12, 5.5))
for i, eixo in enumerate(eixos.ravel()):
    eixo.imshow(x_treino[i * 100])
    eixo.set_title(nomes_classes[y_treino[i * 100]])
    eixo.axis("off")
plt.tight_layout()
plt.show()
```

PASSO 2 · PREPARAR OS DADOS

Dividir por 255

```
x_treino_n = x_treino.astype("float32") / 255.0
x_teste_n = x_teste.astype("float32") / 255.0

print(x_treino_n.min(), x_treino_n.max())
```

Saída esperada: 0.0 1.0 (o menor e o maior valor depois da divisão)

- É a mesma divisão por 255 usada no MNIST: os valores ficam entre 0 e 1
- astype("float32") usa metade da memória: as imagens de treino passam de cerca de 393 MB (float64) para cerca de 197 MB
- A mesma preparação será aplicada às fotos testadas mais adiante

PASSO 3 · MONTAR O MODELO

A rede convolucional para fotos coloridas

```
from tensorflow import keras
from tensorflow.keras import layers

modelo = keras.Sequential([
    keras.Input(shape=(128, 128, 3)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(2, activation="softmax"),
])
```

Camada	O que faz
Input(128, 128, 3)	foto colorida
Conv2D(32, 64, 128)	filtros 3 × 3
MaxPooling2D	reduz a imagem
Flatten	estica em fila
Dense(2, softmax)	gato ou cachorro

PASSO 3 · MONTAR O MODELO

O caminho da imagem pela rede

A imagem encolhe a cada bloco, enquanto o número de filtros aumenta.

Entrada	Conv2D 32	Pooling	Conv2D 64	Pooling	Conv2D 128	Pooling	Flatten	Dense 2
128×128×3	126×126×32	63×63×32	61×61×64	30×30×64	28×28×128	14×14×128	25.088	2

O Flatten gera 25.088 valores (14 × 14 × 128), que a camada final converte em 2 probabilidades: gato e cachorro.

PASSO 3 · MONTAR O MODELO

Quantos parâmetros a rede aprende

Rede	Entrada	Parâmetros
Rede densa (MNIST)	28 × 28	101.770
CNN (MNIST)	28 × 28 × 1	34.826
CNN (gatos e cachorros)	128 × 128 × 3	143.426

- A terceira Conv2D (73.856) e a camada final (50.178) concentram cerca de 86% dos parâmetros
- São 143.426 parâmetros para 1000 imagens de treino: há mais números a aprender do que imagens

Muitos parâmetros e poucas imagens podem favorecer a memorização. Isso será testado adiante.

PASSOS 4 E 5 · COMPILAR E TREINAR

Treinar a rede

```
modelo.compile(optimizer="adam",
               loss="sparse_categorical_crossentropy",
               metrics=["accuracy"])

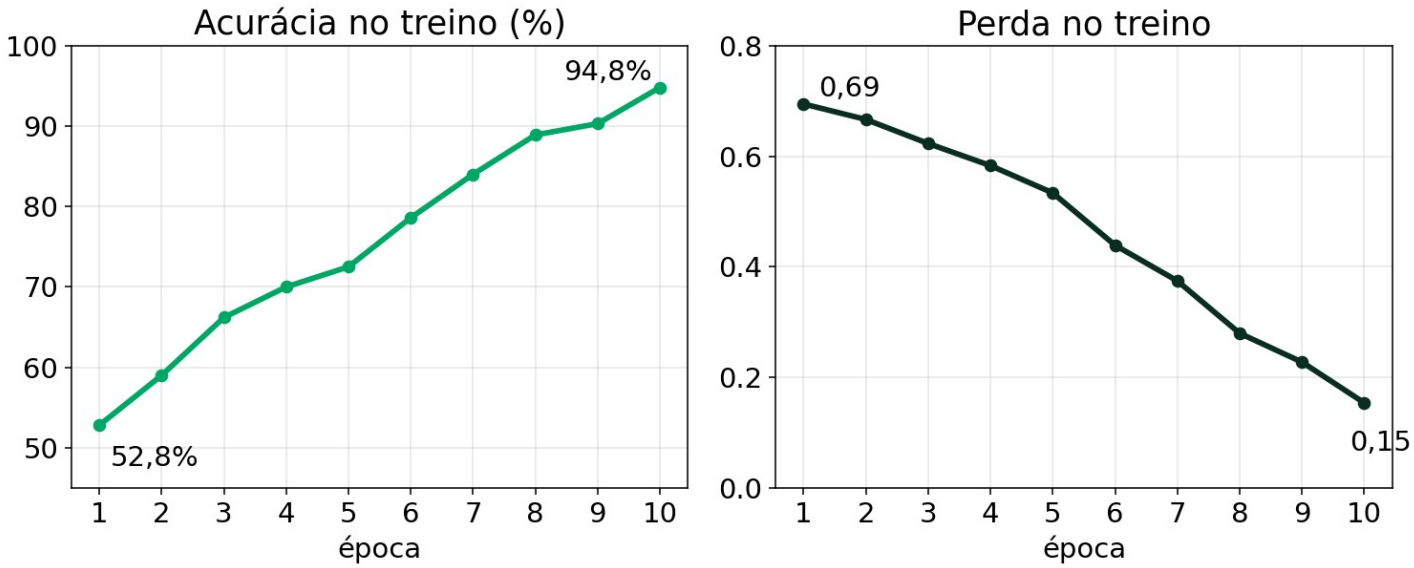
modelo.fit(x_treino_n, y_treino, epochs=10)
```

No Colab do professor, em CPU: cerca de 28 s por época e 5 min 20 s no total.

- A compilação é a mesma do MNIST
- 10 épocas: cada uma passa por todas as 1000 imagens de treino
- A saída mostra a perda (loss) e a acurácia (accuracy) a cada época
- A linha final (History) é só o objeto devolvido pelo fit; é inofensiva

PASSOS 4 E 5 · COMPILAR E TREINAR

Resultado do treino



- Acurácia no treino: de 52,8% para 94,8%
- Perda no treino: de 0,69 para 0,15
- Cerca de 28 s por época (até 41 s)

Execução do professor, em CPU.

A rede aprendeu o conjunto de treino. Falta saber como ela se sai em imagens novas.

PASSO 6 · AVALIAR

Treino não é teste: o que é overfitting

- Treinar é ajustar a rede às imagens de treino, mas o objetivo é acertar imagens novas
- **Overfitting** (sobreajuste): a rede decora as imagens de treino e erra imagens novas
- É como o aluno que decora a lista de exercícios e vai mal na prova, com questões novas
- Para detectar, compare a acurácia no treino com a acurácia no teste

A acurácia no treino (94,8%) ainda não diz nada sobre imagens novas.

PASSO 6 · AVALIAR

Avaliar no teste

```
perda, acuracia = modelo.evaluate(x_teste_n, y_teste, verbose=0)
print(f"Acurácia no teste: {acuracia:.2%}")
```

- evaluate mede o desempenho nas 400 imagens de teste, que a rede não viu
- Anote a acurácia no teste e compare com a do treino

Treino (época 10)

94,8%

Piso (chutar)

50%

PASSO 6 · AVALIAR

Como ler as duas acurácias

Treino	Teste	Interpretação
alto	alto	a rede aprendeu a diferenciar gatos e cachorros
alto	bem menor	overfitting: a rede decorou os exemplos de treino
baixo	baixo	a rede ainda não aprendeu: mais épocas ou outra arquitetura
qualquer	próximo de 50%	desempenho de chute

Compare sempre as duas acurácias antes de concluir que o modelo é bom.

PASSO 7 · PREVER

Escolher as fotos da internet

Boas escolhas

- Um só animal na foto
- O animal ocupa boa parte da imagem
- Foto nítida e bem iluminada
- Fundo simples

Evitar

- Vários animais na mesma foto
- Animal pequeno ao fundo
- Desenhos e ilustrações
- Fotos escuras ou tremidas

Hipótese a testar: fotos mais parecidas com as do treino tendem a dar melhores resultados.

PASSO 7 · PREVER

Enviar a foto e prever

```
import io
import numpy as np
from PIL import Image, ImageOps
import matplotlib.pyplot as plt
from google.colab import files


enviado = files.upload()          # abre a janela para escolher a foto
nome = list(enviado.keys())[0]
foto = ImageOps.exif_transpose(Image.open(io.BytesIO(enviado[nome]))).convert("RGB")


entrada = np.array(foto.resize((128, 128))).astype("float32") / 255.0
probabilidades = modelo.predict(entrada.reshape(1, 128, 128, 3), verbose=0)[0]
classe = int(np.argmax(probabilidades))
nomes_classes = ["gato", "cachorro"]


plt.imshow(foto)
plt.title(f"O modelo acha que é: {nomes_classes[classe]} ({probabilidades[classe]:.0%} de confiança)")
plt.axis("off")
plt.show()
```

A foto é redimensionada para 128 × 128 e dividida por 255, exatamente como as imagens de treino.

PASSO 7 · PREVER

Por que o mesmo preparo do treino

- A rede só conhece imagens de 128 × 128 pixels, coloridas e com valores de 0 a 1
- Com outro tamanho, o Keras acusa erro
- Com outra escala (0 a 255), não há erro, mas a resposta tende a perder o sentido
- Por isso a foto passa pelos mesmos passos das imagens de treino

Regra: na hora de prever, a rede deve receber o mesmo tipo de entrada que recebeu no treino.

ATIVIDADE 1

Testar 10 fotos da internet

1. Escolher na internet 5 fotos de gatos e 5 de cachorros
2. Testar cada uma na célula de previsão
3. Registrar os resultados no quadro de testes
4. Contar os acertos: quantos de 10?
5. Comparar com a acurácia no teste do notebook
6. Escrever uma hipótese para cada erro

ATIVIDADE 2 (OPCIONAL)

Variar a rede e comparar

Variação	O que mudar	Treino	Teste	Tempo por época
Original	epochs=10	94,8%	*a preencher*	cerca de 28 s
Mais épocas	epochs=20			
Menos filtros	16, 32 e 64 nas Conv2D			
Sem o 3º bloco	remover a 3ª Conv2D e o 3º MaxPooling2D			

Cada variação exige treinar de novo, o que leva alguns minutos em CPU. Anote a acurácia no treino, no teste e o tempo por época.

PRÓXIMOS PASSOS

Como melhorar o resultado

Aumento de dados

Criar variações das imagens de treino (espelhar, girar, ampliar) para a rede ver mais exemplos

Transferência de aprendizado

Reaproveitar uma rede já treinada com milhões de imagens e adaptá-la ao novo problema

Os dois ficam para depois: são caminhos usuais quando há poucos dados.

GLOSSÁRIO

Termos novos da aula

Termo	Significado
Arquivo .zip	reúne várias imagens compactadas
Redimensionar	dar a todas as imagens o mesmo tamanho
RGB	3 canais de cor: vermelho, verde e azul
Acurácia no treino	acertos nas imagens usadas para aprender
Acurácia no teste	acertos em imagens que a rede nunca viu

Termo	Significado
Overfitting	a rede decora o treino e erra imagens novas
Confiança	a probabilidade que a rede dá à resposta
Piso (chute)	50% com 2 classes equilibradas
Aumento de dados	criar variações das imagens de treino
Transferência de aprendizado	reaproveitar uma rede treinada em outro problema

RESUMO

Resumo da aula

- A receita do Keras é a mesma; mudaram o carregamento dos dados e a lista de camadas
- Com imagens em arquivos, o nome informa a classe; todas precisam ter o mesmo tamanho e 3 canais
- A CNN de 143.426 parâmetros chegou a 94,8% de acurácia no treino em 10 épocas
- A acurácia no teste é a que mede o desempenho em imagens novas
- Fotos da internet mostram se a rede funciona fora do conjunto de dados

Próximo tema: aprendizado não supervisionado, em que os dados não têm a resposta certa.

PRÓXIMOS PASSOS

Do rótulo ao agrupamento

Até aqui, todos os dados tinham a resposta certa. A seguir, o algoritmo procura padrões sozinho, sem rótulos: K-Means, DBSCAN e PCA.

Atividade opcional: variar a rede e comparar os resultados